Name: MOHAMED MOWLID ALI

Prompt Engineering for software Engineering Task.

Exercise A : Explaining the SDLC.

Prompt 1: Zero shot

- Explain what a Software Development Life cycle is?

AI Response

What is the Software Development Life Cycle (SDLC)?
Software Development Life Cycle (SDLC) is a step-by-step process used to plan, create, test, deploy, and maintain software.

In simple words, SDLC is like a roadmap for building software. It helps developers create software systematically, reduce mistakes, control costs, and make sure the final product meets users' needs.

Main stages of SDLC

1. Planning

Identify the problem to solve.
Determine the goals, resources, time, and budget.
Example: A school wants a system to manage student records.

2. Requirements Analysis

Find out what users need the software to do.
Developers gather requirements from customers, teachers, students, managers, etc.
Example: The system should register students, record marks, and produce reports.

3. Design

Decide how the software will look and work.
This includes designing the database, user interface, system architecture, and technologies.

4. Development (Coding)

Programmers write the actual code.
For example, they might use Python, Java, JavaScript, PHP, or C#.

5. Testing

The software is checked for errors and problems.
Developers test whether each feature works correctly and fix bugs.

6. Deployment

The completed software is released for users.

For example, a school installs the student-management system and starts using it.

7. Maintenance

Developers continue to fix bugs, improve security, and add new features after release.

Example: Adding a new feature that allows parents to view students' results online.

Simple example

Imagine you are developing a school management system:

Plan → Gather requirements → Design → Code → Test → Deploy → Maintain

So, the SDLC ensures that software is not simply coded randomly; it is developed through an organized process from idea to working product and ongoing improvement.

Summary

Software Development Life Cycle (SDLC) is a step-by-step process used to develop software systematically. It includes planning, requirements analysis, design, development, testing, deployment, and maintenance. SDLC helps developers create reliable software that meets users' needs while reducing errors and costs.

Prompt 2:Role prompting

You are a senior Software engineer, explain what a Software Development Life Cycle is to a beginner software developer?. Answering in one short paragraph.

AI Response

Software Development Life Cycle (SDLC) is a step-by-step process used by software developers to plan, design, build, test, deploy, and maintain software. It helps teams organize their work, reduce errors, control costs, and ensure the software meets users' needs. The main stages are planning, requirements analysis, design, development, testing, deployment, and maintenance.

Summary

Software Development Life Cycle (SDLC) is a structured process used to develop software from an idea to a finished product. It includes stages such as planning, requirements, design, development, testing, deployment, and maintenance. SDLC helps developers create reliable, high-quality software that meets users' needs.

Comparing the two prompts

- The second response was more useful because it has a shorter, clearer message that is suited to a software beginner.
- I changed the prompt by adding a professional role[senior software engineer] and a target audience [beginner software developer].
- The role encouraged a more professional and accurate explanation, while the audience made the explanation simpler, clearer, and easier for a beginner to understand.

EXERCISE B: Software Development models

Prompt: You are a senior software engineer. For each of the following four project situations, recommend
the most appropriate software development model, explain why it is suitable, and briefly mention one limitation of the model.

AI Response.

Situation	Recommended model	Explanation & Limitation
Stable, clearly defined requirements	Waterfall Model	Suitable because requirements are fixed and well understood, allowing development to proceed through clearly defined stages. Limitation: It is difficult to accommodate changes once development has started.
Startup with frequently changing requirements	Agile model	Suitable because Agile supports frequent changes, customer feedback, and continuous improvements through short development cycles. Limitation: Changing requirements can make planning and budgeting

		difficult.
Large project with technical and security risks	Spiral Model	Suitable because it focuses strongly on identifying and reducing risks through repeated planning, development, and evaluation. Limitation: It can be expensive and complex to manage.
Testing and verification are extremely important	V-Model	Suitable because each development stage is closely linked to a corresponding testing and verification stage. Limitation: It is less flexible when requirements change.

Reflection

I learned that the best software development model depends on the characteristics and needs of the project. Stable requirements are suitable for models such as Waterfall, while changing requirements benefit from Agile. Projects with high risks may require the Spiral model, while projects where testing and verification are critical can use the V-Model.

Exercise C: Requirements Extraction

Prompt: You are a senior software requirements analyst. Analyze the following client brief for a local bookstore.
Clearly explain your reasoning and avoid adding requirements that are not supported by the brief.

AI Response

1. Functional Requirements
   - The system should track books, quantities, and prices.
   - The system should manage supplier payments.
   - The system should alert staff when stock is low.
   - The system should identify best-selling books.
   - Customers should be able to check book availability online.
   - The system should produce clear records for the part-time assistant.

2. Non-Functional Requirements
   - ☐ The system should be easy to use, especially for the part-time assistant.
   - ☐ The system should be affordable because the owner has a limited budget.

- ☐ The online availability information should be reasonably accurate and up to date.
- ☐ The system should provide reliable records.

3. Must Have Requirements
   - ➔ Track books, quantities, and prices.
   - ➔ Manage supplier payments.
   - ➔ Provide low-stock alerts.
   - ➔ Produce clear records.
   - ➔ Allow customers to check availability online.

4. Should Have Requirements
   - Identify best-selling books.
   - Provide an easy-to-use interface.
   - Keep availability information up to date.

5. Nice to Have Requirements
   - Custom reports and dashboards.
   - Advanced sales analysis.
   - Automatic notifications to customers.

6. Questions or Ambiguities
   - What number or percentage should trigger a low-stock alert?
   - What types of supplier payments need to be recorded?
   - How often should best-selling books be calculated?
   - Should customers be able to buy books online, or only check availability?
   - What type of records does the part-time assistant need?
   - What is the available budget?
   - Who will update book quantities and prices?

Evaluate

Two things the AI did well

1. It separated functional and non-functional requirements clearly. For example, tracking books is a functional requirement, while ease of use and affordability are non-functional concerns.
2. It identified important clarification questions. For example, asking what stock level should trigger an alert helps prevent misunderstandings during development.

One thing it missed or categorized incorrectly

The AI placed "Identify best-selling books" under Should Have, but the client explicitly said the software needs to identify best-selling books. Therefore, it could reasonably be considered a Must Have requirement.
One improvement I would make
I would improve the prompt by asking the AI to justify why each requirement is classified as Must Have, Should Have, or Nice to Have, while also considering the owner's limited budget. This would make the prioritization more accurate and useful for planning the software.

Exercise D: Prompt Iteration Challenge

Version 1

Prompt: Create a simple project plan for a mobile application that allows university students to book campus study rooms.

AI Response

- ➔ The project plan should include:
- ➔ Planning
- ➔ Designing the application
- ➔ Developing the application
- ➔ Testing
- ➔ Launching the application
- ➔ Maintaining the application

What is Missing?

The response does not provide specific SDLC phases, time estimates, deliverables, risks, risk mitigation, team roles, or a budget.

Version 2

Prompt: You are a software project manager. Create a project plan for a mobile application that allows university students to book campus study rooms. The application should allow students to view available rooms, select a date and time, make a booking, and receive booking confirmation. Include the main development activities and explain what should be done at each stage.

What I Changed

I added a role, more information about the application, and specific features such as viewing rooms, booking times, and receiving confirmation.

Why?

This gives the AI more context about the project and helps it create a more relevant project plan.

Version 3

Prompt: You are a senior software project manager. Develop a detailed project plan for a mobile application that allows university students to book campus study rooms. Students should be able to register or log in, view available rooms, check room capacity, select a date and time, make or cancel bookings, and receive confirmation.

Organize the plan according to the Software Development Life Cycle (SDLC). For each phase, describe the main activities and expected outputs. Also identify possible project challenges and explain how they could be addressed. Assume the project is being developed by a small university software team with a limited budget**.**

What I Changed

I added more functional requirements, specified the SDLC, identified the team and budget situation, and asked for challenges and solutions.

Why?

These details help the AI produce a more realistic and useful plan instead of giving only a general list of activities.

Version 4: Final prompt

Final prompt: You are a senior software project manager and software engineer. Create a realistic project plan for a mobile application that allows university students to book campus study rooms.
The application should allow students to:

- Register and log in.

- View available study rooms.
- Check room capacity and facilities.
- Select a date and time.
- Book or cancel a room.
- Receive booking confirmation.
- View their upcoming bookings.

Assume the project is being developed by a small university software team with a limited budget.
Organize the project plan around all 7 SDLC phases:

1. Planning
2. Requirements Analysis
3. System Design
4. Development/Implementation
5. Testing
6. Deployment
7. Maintenance

For each SDLC phase, provide:

- Main activities
- Estimated time
- Key deliverables
- Team members involved

Also include:

- Major project risks
- Risk mitigation strategies
- Team roles and responsibilities
- A rough project budget
- Assumptions made
- A short overall project timeline

Present the answer in clear tables where appropriate. Keep the estimates realistic for a small university project and clearly state that the budget and time estimates are approximate**.**

AI Response

SDLC Phase	Main Activities	Time	Key Deliverables
1. Planning	Define goals, scope, users, resources, and schedule	1 week	Project plan
2. Requirements Analysis	Gather student and administrator requirements	2 weeks	Requirements specification
3.System Design	Design user interface, database, and system architecture	2 weeks	UI designs and system design
4. Development	Build login, room availability, booking, cancellation, and notifications	6 weeks	Working mobile application
5. Testing	Functional, usability, security, and performance testing	2 weeks	Test reports
6.Deployment	Prepare production environment and release the application	1 week	Deployed application
7. Maintenance	Fix bugs, monitor performance, and provide updates	Ongoing	Updates and maintenance reports

Team Roles

Project Manager: Plans and coordinates the project.

- Business/Requirements Analyst: Collects and documents requirements.
- UI/UX Designer: Designs the application interface.
- Mobile Developer: Develops the mobile application.
- Backend Developer: Develops the server and database.
- QA Tester: Tests the application and reports defects.
- System Administrator/DevOps: Handles deployment and infrastructure.

Major Risks and Mitigation

Risk	Mitigation
Requirements change	Confirm requirements early and use change

	control.
Security or data privacy problems	Use secure authentication and protect student data.
Development delays	Use milestones and monitor progress regularly.
Booking conflicts	Use database validation to prevent double bookings.
Limited budget	Prioritize essential features and use suitable low-cost technologies.

Rough Budget

Item	Estimated Cost
Development and testing	$8,000
Hosting and database	$1,000
Design and software tools	$500
Testing and deployment	$500
Contingency	$1,000
Estimated Total	$11,000

Overall estimated development time: approximately 15 weeks, followed by ongoing maintenance.

Final Reflection

Yes, the prompt improved significantly from Version 1 to Version 4. The biggest difference was adding specific project requirements, all seven SDLC phases, time estimates, deliverables, risks, mitigation strategies, team roles, and budget requirements. These details made the AI response much more specific, realistic, and useful.

Overall Reflection

1. What makes a prompt effective?

An effective prompt is clear, specific, and provides enough context. It should clearly explain what the AI needs to produce and how the answer should be organized.

2. Which prompting technique did you find most useful?

I found role prompting and adding detailed context most useful because they helped the AI respond as a software professional and produce a more relevant answer.

3. Why is context important when working with AI?

Context helps the AI understand the situation, requirements, users, limitations, and expected outcome. Without context, the response may be too general.

4. Why should we review AI-generated responses instead of accepting them automatically?

AI can make mistakes, misunderstand requirements, or provide unrealistic estimates. Reviewing the response helps identify errors and ensures that the information is appropriate for the project.

5. How can AI support software engineers across the SDLC?

AI can help engineers gather and analyze requirements, generate designs and code, create test cases, identify risks, document systems, troubleshoot problems, and support maintenance.